

Letter of Credit Workflow + Reusable Agents Plan

Goal

Create one Import LC workflow (WF-LC) with processes, canvas node-to-node connections, and 6 reusable Functional Agents, seeded in DB so FE pages already render them: Agent Library, Workflow list, Drag-Drop designer, and evaluation screens.

How this fits the current platform

[Mermaid diagram omitted - see original plan markdown]

Org strategy (locked): Seed on organization_id = 14 (same as Cheque Book / Card Dispatch). clone_org_demo.py already clones Ujjivan panel data onto AU Small Finance Bank Limited (admin Hasinirady@openlayer.technology). Do not invent a separate AU-only workflow ID path.

Reference pattern: cheque_book_workflow.sql + cheque_book_agents_seed.sql + process_templates.py + seed_canvases.py.

FE code changes: none required for first render ? data-driven. Existing routes already cover library/designer/evaluation (App.jsx).

Workflow definition (WF-LC)

Name: Letter of Credit (Import) Workflow

Code: W

Phase | Process | Assigned reusable agent | Human / branch

P01 | LC F

High-level agent sequence matches the document:

[Mermaid diagram omitted - see original plan markdown]

6 reusable Functional Agents (catalog)

Seed into workflow_ai_agents with existing master categories/subcategories (not LC-only Industry-Specific), so they appear in Agent Library and can be dropped onto any workflow canvas.

Reuse rules in seed metadata:

- agent_

Document types from the LC doc (aligned to Document Reconciliation intent): Invoice, Bill of Lading, Certificate, Insurance ? store as agent capabilities / knowledge_base text on Document Intelligence + Exception agents (no separate PDF parse required for v1).

Org systems + APIs to add (org 14)

Cheque Book systems already exist (CBS, eKYC, Risk, Notification). LC needs additional org systems + integrations (seed idempotently):

Systems: Trade Finance System / Letter of Credit System, SWIFT Gateway, AML Monitoring System, Sanctions Screening System, Credit Risk System (and reuse existing CRM/CBS/eKYC/Notification).

APIs (examples): LC Issuance API, SWIFT MT700 Send API, AML Screen API, Sanctions Screen API, Credit Limit API, Trade Document Upload API, Settlement API, Reconciliation API.

Files: new letter_of_credit_systems_seed.sql joining existing organization_system_systems.sql / system_integrations.sql patterns.

Canvas (node-to-node)

1. Extend process_templates.py:

- Keep

2. After SQL seed, call seed_empty_process_canvases(db, workflow_code="WF-LC") from seed.py (and ensure clone path also fills empty canvases ? already does).

3. Sub-workflow design_json initially empty nodes/edges arrays (like Cheque Book); Python canvas seeder fills real React Flow nodes/edges so DragDropDesigner shows connections immediately.

Runtime stubs (reuse-ready)

Create thin Functional Agent classes under:

FT-OPEN-LAYER-BE/app/ai_runtime/functional_agents/reusable/

- customer_compliance_agent.py

- trade_r

Each subclasses existing BaseAgent / follows Cheque Book stub style: recommend/validate only; regulated decisions stay HITL. Register via agent_class_path in catalog key_capabilities. No Temporal / full SWIFT integration in v1.

Files to create / change

Create

- FT-OP
17 sub_y

Modify

- FT-OP
run, call

Do not change for v1: process_evaluation / workflow_evaluation master rubrics (already seeded); FE pages (data-driven).

Verification (after implementation)

1. Run backend seed / restart so incremental SQL + canvas seed execute.

2. API: v

Out of scope (v1)

- Live SWIFT/CBS connectors beyond stub allowlists

- Full Do